

DBMS Assignment 11

Tables:

Classroom table:

```
mysql> select * from classroom;
+-----+-----+-----+
| building | room_number | capacity |
+-----+-----+-----+
| Packard  | 101         | 500      |
| Painter  | 514         | 10       |
| Taylor   | 3128        | 70       |
| Watson   | 100         | 30       |
| Watson   | 120         | 50       |
+-----+-----+-----+
5 rows in set (0.01 sec)
```

Department table:

```
mysql> select * from department;
+-----+-----+-----+
| dept_name | building | budget |
+-----+-----+-----+
| Biology   | Watson   | 90000.00 |
| Comp. Sci. | Taylor   | 100000.00 |
| Elec. Eng. | Taylor   | 85000.00 |
| Finance    | Painter  | 120000.00 |
| History    | Painter  | 50000.00 |
| Music      | Packard  | 80000.00 |
| Physics    | Watson   | 70000.00 |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

Course table:

```
mysql> select * from course;
+-----+-----+-----+-----+
| course_id | title | dept_name | credits |
+-----+-----+-----+-----+
| BIO-101   | Intro. to Biology | Biology | 4 |
| BIO-301   | Genetics | Biology | 4 |
| BIO-399   | Computational Biology | Biology | 3 |
| CS-101    | Intro. to Computer Science | Comp. Sci. | 4 |
| CS-190    | Game Design | Comp. Sci. | 4 |
| CS-315    | Robotics | Comp. Sci. | 3 |
| CS-319    | ImageProcessing | Comp. Sci. | 3 |
| CS-347    | DatabaseSystem Concepts | Comp. Sci. | 3 |
| EE-181    | Intro. toDigital Systems | Elec. Eng. | 3 |
| FIN-201   | Investment Banking | Finance | 3 |
| HIS-351   | WorldHistory | History | 3 |
| MU-199    | Music VideoProduction | Music | 3 |
| PHY-101   | Physical Principles | Physics | 4 |
+-----+-----+-----+-----+
13 rows in set (0.00 sec)
```

instructor table:

```
mysql> select * from instructor;
```

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

11 rows in set (0.00 sec)

Section table:

```
mysql> select * from section;
```

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

15 rows in set (0.00 sec)

Teaches table:

```
mysql> select * from teaches;
```

ID	course_id	sec_id	semester	year
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
10101	CS-101	1	Fall	2009
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
10101	CS-315	1	Spring	2010
83821	CS-319	2	Spring	2010
10101	CS-347	1	Fall	2009
98345	EE-181	1	Spring	2009
12121	FIN-201	1	Spring	2010
32343	HIS-351	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

13 rows in set (0.00 sec)

student table:

```
mysql> select * from student;
```

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

```
13 rows in set (0.00 sec)
```

takes table:

```
mysql> select * from takes;
```

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	NULL

```
22 rows in set (0.00 sec)
```

advisor table:

```
mysql> select * from advisor;
+-----+-----+
| s_ID | i_ID |
+-----+-----+
| 12345 | 10101 |
| 44553 | 22222 |
| 45678 | 22222 |
| 23121 | 76543 |
| 98988 | 76766 |
| 76653 | 98345 |
| 98765 | 98345 |
+-----+-----+
7 rows in set (0.00 sec)
```

time_slot table:

```
mysql> select * from time_slot;
+-----+-----+-----+-----+-----+-----+
| time_slot_id | day | start_hr | start_min | end_hr | end_min |
+-----+-----+-----+-----+-----+-----+
| A | F | 8 | 0 | 8 | 50 |
| A | M | 8 | 0 | 8 | 50 |
| A | W | 8 | 0 | 8 | 50 |
| B | F | 9 | 0 | 9 | 50 |
| B | M | 9 | 0 | 9 | 50 |
| B | W | 9 | 0 | 9 | 50 |
| C | F | 11 | 0 | 11 | 50 |
| C | M | 11 | 0 | 11 | 50 |
| C | W | 11 | 0 | 11 | 50 |
| D | F | 13 | 0 | 13 | 50 |
| D | M | 13 | 0 | 13 | 50 |
| D | W | 13 | 0 | 13 | 50 |
| E | R | 10 | 30 | 11 | 45 |
| E | T | 10 | 30 | 11 | 45 |
| F | R | 14 | 30 | 15 | 45 |
| F | T | 14 | 30 | 15 | 45 |
| G | F | 16 | 0 | 16 | 50 |
| G | M | 16 | 0 | 16 | 50 |
| G | W | 16 | 0 | 16 | 50 |
| H | W | 10 | 0 | 12 | 30 |
+-----+-----+-----+-----+-----+-----+
20 rows in set (0.00 sec)
```

prereq table:

```
mysql> select * from prereq;
+-----+-----+
| course_id | prereq_id |
+-----+-----+
| BIO-301 | BIO-101 |
| BIO-399 | BIO-101 |
| CS-190 | CS-101 |
| CS-315 | CS-101 |
| CS-319 | CS-101 |
| CS-347 | CS-101 |
| EE-181 | PHY-101 |
+-----+-----+
7 rows in set (0.00 sec)
```

Question1:

a)

```
mysql> SELECT title
-> FROM course
-> WHERE dept_name = 'Comp. Sci.'
-> AND credits = 3;
+-----+
| title |
+-----+
| Robotics |
| Image Processing |
| Database System Concepts |
+-----+
3 rows in set (0.01 sec)
```

b)

```
mysql> SELECT DISTINCT t.ID
-> FROM takes t
-> JOIN teaches te ON t.course_id = te.course_id
-> AND t.sec_id = te.sec_id
-> AND t.semester = te.semester
-> AND t.year = te.year
-> JOIN instructor i ON te.ID = i.ID
-> WHERE i.name = 'Einstein';
+-----+
| ID |
+-----+
| 44553 |
+-----+
```

c)

```
mysql> SELECT MAX(salary) AS highest_salary
-> FROM instructor;
+-----+
| highest_salary |
+-----+
| 95000.00 |
+-----+
1 row in set (0.01 sec)
```

d)

```
mysql> SELECT ID, name, salary
-> FROM instructor
-> WHERE salary = (SELECT MAX(salary) FROM instructor);
+-----+-----+-----+
| ID | name | salary |
+-----+-----+-----+
| 22222 | Einstein | 95000.00 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

e)

```
mysql> SELECT s.course_id, s.sec_id, COUNT(t.ID) AS enrollment
-> FROM section s
-> JOIN takes t ON s.course_id = t.course_id
->                AND s.sec_id = t.sec_id
->                AND s.semester = t.semester
->                AND s.year = t.year
-> WHERE s.semester = 'Fall'
->        AND s.year = 2009
-> GROUP BY s.course_id, s.sec_id;
```

course_id	sec_id	enrollment
CS-101	1	6
CS-347	1	2
PHY-101	1	1

```
3 rows in set (0.01 sec)
```

f)

```
mysql> SELECT MAX(enrollment) AS max_enrollment
-> FROM (
->     SELECT COUNT(t.ID) AS enrollment
->     FROM section s
->     JOIN takes t ON s.course_id = t.course_id
->                 AND s.sec_id = t.sec_id
->                 AND s.semester = t.semester
->                 AND s.year = t.year
->     WHERE s.semester = 'Fall'
->           AND s.year = 2009
->     GROUP BY s.course_id, s.sec_id
-> ) AS section_enrollments;
```

max_enrollment
6

```
1 row in set (0.01 sec)
```

g)

```
mysql> SELECT s.course_id, s.sec_id, COUNT(t.ID) AS enrollment
-> FROM section s
-> JOIN takes t ON s.course_id = t.course_id
->                AND s.sec_id = t.sec_id
->                AND s.semester = t.semester
->                AND s.year = t.year
-> WHERE s.semester = 'Fall'
->        AND s.year = 2009
-> GROUP BY s.course_id, s.sec_id
-> HAVING COUNT(t.ID) = (
->     SELECT MAX(enrollment) AS max_enrollment
->     FROM (
->         SELECT COUNT(t.ID) AS enrollment
->         FROM section s
->         JOIN takes t ON s.course_id = t.course_id
->                 AND s.sec_id = t.sec_id
->                 AND s.semester = t.semester
->                 AND s.year = t.year
->         WHERE s.semester = 'Fall'
->         AND s.year = 2009
->         GROUP BY s.course_id, s.sec_id
->     ) AS section_enrollments
-> );
```

course_id	sec_id	enrollment
CS-101	1	6

1 row in set (0.00 sec)

Question2:

a)

```
mysql> UPDATE instructor
-> SET salary = salary * 1.10
-> WHERE dept_name = 'Comp. Sci.';
Query OK, 2 rows affected (0.02 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

b)

```
mysql> DELETE FROM course
-> WHERE course_id NOT IN (SELECT DISTINCT course_id FROM section);
Query OK, 1 row affected (0.01 sec)
```

c)

```
mysql> INSERT INTO instructor (ID, name, dept_name, salary)
-> SELECT ID, name, dept_name, 30000
-> FROM student
-> WHERE tot_cred > 100;
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Question3:

```
mysql> SELECT dept_name
-> FROM department
-> WHERE LOWER(dept_name) LIKE '%sci%';
+-----+
| dept_name |
+-----+
| Comp. Sci. |
+-----+
1 row in set (0.01 sec)
```

Question 4:

a)

```
mysql> SELECT i.ID, i.name, COUNT(t.course_id) AS num_sections
-> FROM instructor i
-> LEFT OUTER JOIN teaches t ON i.ID = t.ID
-> GROUP BY i.ID, i.name;
+-----+-----+-----+
| ID | name | num_sections |
+-----+-----+-----+
| 00128 | Zhang | 0 |
| 10101 | Srinivasan | 3 |
| 12121 | Wu | 1 |
| 15151 | Mozart | 1 |
| 22222 | Einstein | 1 |
| 23121 | Chavez | 0 |
| 32343 | El Said | 1 |
| 33456 | Gold | 0 |
| 58583 | Califieri | 0 |
| 76543 | Singh | 0 |
| 76766 | Crick | 2 |
| 83821 | Brandt | 3 |
| 98345 | Kim | 1 |
| 98988 | Tanaka | 0 |
+-----+-----+-----+
14 rows in set (0.00 sec)
```

b)

```
mysql> SELECT i.ID, i.name,
-> (SELECT COUNT(*)
-> FROM teaches t
-> WHERE t.ID = i.ID) AS num_sections
-> FROM instructor i;
+-----+-----+-----+
| ID | name | num_sections |
+-----+-----+-----+
| 00128 | Zhang | 0 |
| 10101 | Srinivasan | 3 |
| 12121 | Wu | 1 |
| 15151 | Mozart | 1 |
| 22222 | Einstein | 1 |
| 23121 | Chavez | 0 |
| 32343 | El Said | 1 |
| 33456 | Gold | 0 |
| 58583 | Califieri | 0 |
| 76543 | Singh | 0 |
| 76766 | Crick | 2 |
| 83821 | Brandt | 3 |
| 98345 | Kim | 1 |
| 98988 | Tanaka | 0 |
+-----+-----+-----+
14 rows in set (0.00 sec)
```

c)


```
mysql> SELECT s.course_id, s.sec_id, s.semester, s.year,
-> COALESCE(i.name, '-') AS instructor_name
-> FROM section s
-> LEFT JOIN teaches t ON s.course_id = t.course_id
-> AND s.sec_id = t.sec_id
-> AND s.semester = t.semester
-> AND s.year = t.year
-> LEFT JOIN instructor i ON t.ID = i.ID
-> WHERE s.semester = 'Spring' AND s.year = 2010;
```

course_id	sec_id	semester	year	instructor_name
CS-101	1	Spring	2010	?
FIN-201	1	Spring	2010	Wu
MU-199	1	Spring	2010	Mozart
HIS-351	1	Spring	2010	El Said
CS-319	2	Spring	2010	Brandt
CS-319	1	Spring	2010	?
CS-315	1	Spring	2010	Srinivasan

7 rows in set (0.00 sec)

d)

```
mysql> SELECT d.dept_name, COUNT(i.ID) AS num_instructors
-> FROM department d
-> LEFT JOIN instructor i ON d.dept_name = i.dept_name
-> GROUP BY d.dept_name;
```

dept_name	num_instructors
Biology	2
Comp. Sci.	3
Elec. Eng.	1
Finance	3
History	2
Music	1
Physics	2

7 rows in set (0.00 sec)

Question 5:

a)

```
mysql> SELECT s.ID, s.name, s.dept_name, s.tot_cred, t.course_id, t.sec_id, t.semester, t.year, t.grade
-> FROM student s
-> LEFT JOIN takes t ON s.ID = t.ID;
```

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
70557	Snow	Physics	0	NULL	NULL	NULL	NULL	NULL
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	NULL

23 rows in set (0.00 sec)

b)

```
mysql> SELECT s.ID, s.name, s.dept_name, s.tot_cred, t.course_id, t.sec_id, t.semester, t.year, t.grade
-> FROM student s
-> LEFT JOIN takes t ON s.ID = t.ID
-> UNION
-> SELECT s.ID, s.name, s.dept_name, s.tot_cred, t.course_id, t.sec_id, t.semester, t.year, t.grade
-> FROM takes t
-> LEFT JOIN student s ON t.ID = s.ID;
```

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
70557	Snow	Physics	0	NULL	NULL	NULL	NULL	NULL
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	NULL

23 rows in set (0.01 sec)

Question 6:

```

mysql> CREATE TABLE BankCustomers (
->     accNum INT PRIMARY KEY,
->     name VARCHAR(100),
->     loan DECIMAL(15, 2) CHECK (loan <= 1000000)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER loan_check_before_insert
-> BEFORE INSERT ON BankCustomers
-> FOR EACH ROW
-> BEGIN
->     IF NEW.loan > 1000000 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Loan amount cannot exceed 10 lakhs.';
->     END IF;
-> END $$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER loan_check_before_update
-> BEFORE UPDATE ON BankCustomers
-> FOR EACH ROW
-> BEGIN
->     IF NEW.loan > 1000000 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Loan amount cannot exceed 10 lakhs.';
->     END IF;
-> END $$
Query OK, 0 rows affected (0.01 sec)

```

Verification:

```

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO BankCustomers (accNum, name, loan)
-> VALUES (1, 'John Doe', 500000); -- Loan is within the valid range
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO BankCustomers (accNum, name, loan)
-> VALUES (2, 'Jane Smith', 1500000); -- Loan exceeds 10 lakhs
ERROR 1644 (45000): Loan amount cannot exceed 10 lakhs.
mysql> |

```

Question 7:

```

mysql> CREATE TABLE BankingCustomers (
->     accNum INT PRIMARY KEY,
->     name VARCHAR(100),
->     loan DECIMAL(12, 2),
->     major BOOLEAN -- Assuming major is a boolean indicating if the customer is a major
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE InsertCustomerAndListMajors(
->     IN p_accNum INT,
->     IN p_name VARCHAR(100),
->     IN p_loan DECIMAL(12, 2),
->     IN p_major BOOLEAN
-> )
-> BEGIN
->     -- Declare a handler for duplicate key errors
->     DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
->     BEGIN
->         -- If duplicate key occurs, exit the procedure
->         SELECT 'Duplicate Key Error: Customer already exists. Terminating procedure.';
->         RESIGNAL; -- Rethrow the exception to stop execution of the procedure
->     END;
->
->     -- Try to insert a new customer into the BankingCustomers table
->     INSERT INTO BankingCustomers (accNum, name, loan, major)
->     VALUES (p_accNum, p_name, p_loan, p_major);
->
->     -- Count and display the number of customers who are majors
->     SELECT COUNT(*) AS NumberOfMajors
->     FROM BankingCustomers
->     WHERE major = TRUE;
-> END $$
Query OK, 0 rows affected (0.03 sec)

```

Calling stored procedure:

```
mysql>
mysql> DELIMITER ;
mysql> CALL InsertCustomerAndListMajors(101, 'Alice Green', 500000.00, TRUE);
+-----+
| NumberOfMajors |
+-----+
| 1 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> CALL InsertCustomerAndListMajors(101, 'John Doe', 400000.00, FALSE);
+-----+
| Duplicate Key Error: Customer already exists. Terminating procedure. |
+-----+
| Duplicate Key Error: Customer already exists. Terminating procedure. |
+-----+
1 row in set (0.00 sec)

ERROR 1062 (23000): Duplicate entry '101' for key 'bankingcustomers.PRIMARY'
mysql> CALL InsertCustomerAndListMajors(102, 'Bob Black', 700000.00, TRUE);
+-----+
| NumberOfMajors |
+-----+
| 2 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Question 8:

Tables:

```
mysql> CREATE TABLE Branch (
->   branch_id INT PRIMARY KEY,
->   branch_name VARCHAR(100),
->   total_asset DECIMAL(12, 2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Depositor (
->   accNum INT,
->   customer_id INT,
->   PRIMARY KEY (accNum, customer_id),
->   FOREIGN KEY (accNum) REFERENCES BankingCustomers(accNum)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
mysql> CREATE TABLE BankingCustomersNew (
->   accNum INT PRIMARY KEY,
->   name VARCHAR(100),
->   loan DECIMAL(15, 2),
->   branch_id INT, -- Assuming each customer belongs to a branch
->   balance DECIMAL(15, 2),
->   address VARCHAR(255),
->   FOREIGN KEY (branch_id) REFERENCES Branch(branch_id) -- Foreign Key linking to Branch table
-> );
Query OK, 0 rows affected (0.04 sec)
```

Values in tables:

```
mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Branch (branch_id, branch_name, total_asset)
-> VALUES
-> (1, 'Main Branch', 5000000.00),
-> (2, 'North Branch', 3000000.00);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO BankingCustomersNew (accNum, name, loan, branch_id, balance, address)
-> VALUES
-> (101, 'Alice Johnson', 100000.00, 1, 5000.00, '123 Main St'),
-> (102, 'Bob Smith', 200000.00, 1, 15000.00, '456 Oak Ave'),
-> (103, 'Charlie Brown', 150000.00, 2, 20000.00, '789 Pine Rd');
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Trigger:

```
mysql> DELIMITER ;
mysql> CREATE TABLE BankingCustomersNew (
-> accNum INT PRIMARY KEY,
-> name VARCHAR(100),
-> loan DECIMAL(15, 2),
-> branch_id INT, -- Assuming each customer belongs to a branch
-> balance DECIMAL(15, 2),
-> address VARCHAR(255),
-> FOREIGN KEY (branch_id) REFERENCES Branch(branch_id) -- Foreign Key linking to Branch table
-> );
Query OK, 0 rows affected (0.04 sec)
```

Verification:

```
mysql> DELETE FROM BankingCustomersNew WHERE accNum = 101;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Branch;
+-----+-----+-----+
| branch_id | branch_name | total_asset |
+-----+-----+-----+
|          1 | Main Branch | 4900000.00 |
|          2 | North Branch | 3000000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Question 9:

Tables:

```

mysql> CREATE TABLE Employee (
->     Emp_id INT PRIMARY KEY,
->     Emp_name VARCHAR(100),
->     experience INT -- years of experience
-> );
ERROR 1050 (42S01): Table 'employee' already exists
mysql> CREATE TABLE employeetable (
->     Emp_id INT PRIMARY KEY,
->     Emp_name VARCHAR(100),
->     experience INT -- years of experience
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE EmployeePosition (
->     Emp_id INT PRIMARY KEY,
->     Emp_name VARCHAR(100),
->     Position VARCHAR(50),
->     FOREIGN KEY (Emp_id) REFERENCES employeetable(Emp_id) ON DELETE CASCADE
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO employeetable (Emp_id, Emp_name, experience)
-> VALUES
-> (1, 'Alice', 30),
-> (2, 'Bob', 15),
-> (3, 'Charlie', 8),
-> (4, 'David', 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

```

Stored function:

```

mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE AssignPositionsToAllEmployees()
-> BEGIN
->     DECLARE done INT DEFAULT 0;
->     DECLARE emp_id INT;
->     DECLARE emp_name VARCHAR(100);
->     DECLARE emp_experience INT;
->     DECLARE cur CURSOR FOR
->         SELECT Emp_id, Emp_name, experience FROM employeetable;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
->
->     OPEN cur;
->
->     read_loop: LOOP
->         FETCH cur INTO emp_id, emp_name, emp_experience;
->
->         IF done THEN
->             LEAVE read_loop;
->         END IF;
->
->         -- Debug check to see the fetched values
->         IF emp_id IS NULL OR emp_name IS NULL OR emp_experience IS NULL THEN
->             SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Null value encountered during cursor fetch';
->         END IF;
->
->         -- Insert into EmployeePosition table using the AssignPosition function
->         INSERT INTO EmployeePosition (Emp_id, Emp_name, Position)
->         VALUES (emp_id, emp_name, AssignPosition(emp_experience));
->     END LOOP;
->
->     CLOSE cur;
-> END $$
Query OK, 0 rows affected (0.01 sec)

```

Question 10:

Trigger:

```

mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER UpdateEmployeePositionAfterInsert
-> AFTER INSERT ON EmployeeTable
-> FOR EACH ROW
-> BEGIN
->     -- Determine the position based on the experience of the new employee
->     DECLARE position VARCHAR(50);
->
->     IF NEW.experience > 25 THEN
->         SET position = 'Senior Executive';
->     ELSEIF NEW.experience > 10 THEN
->         SET position = 'Executive';
->     ELSEIF NEW.experience > 5 THEN
->         SET position = 'Senior Analyst';
->     ELSE
->         SET position = 'Analyst';
->     END IF;
->
->     -- Insert the new employee's position into the EmployeePosition table
->     INSERT INTO EmployeePosition (Emp_id, Emp_name, Position)
->     VALUES (NEW.Emp_id, NEW.Emp_name, position);
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;

```

Verification:

```

mysql> -- Insert a new employee into the EmployeeTable
mysql> INSERT INTO EmployeeTable (Emp_id, Emp_name, experience)
-> VALUES (101, 'John Doe', 12);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Check the EmployeePosition table
mysql> SELECT * FROM EmployeePosition;
+-----+-----+-----+
| Emp_id | Emp_name | Position |
+-----+-----+-----+
| 101 | John Doe | Executive |
+-----+-----+-----+
1 row in set (0.00 sec)

```